

IoT - PLC Team Case Study Solution

ITB de Labo 2026 Recruitment

Name: Muhammad Ammar Hanif

Major: Electrical Engineering (Batch 2024)

Student ID (*Nomor Induk Mahasiswa*): 13224087

Context: I am only providing answer for **Problem 3 (Smart Control Showroom: 3D Digital Twin)**. So the document content is fully **Problem 3 Answer**.

[Report & Final Web App Version Demonstration Link](#)

3. Smart Control Showroom: 3D Digital Twin

3.1 Device Orchestration & Unified CCTV. **Design an orchestration layer that abstracts different communication protocols behind a single command interface.**

a. System Stack

Technology Stack Needed:

- System Design: **Facade Design Pattern**. For the illustration, look forward to **Appendix 4** and **Appendix 5**.
- Backend: **Node.js (Express.js)** [5]
- Frontend: **React.js** (for 3D requirements, it will be using react-three-fiber, which includes and useGLTF + GLTFLoader + DRACOLoader + OrbitControls (user drag/zoom of the 3D viewer) + Tailwind CSS (UI). [6]
- Communication Protocols: **Cloud API, Local API, BLE, and MQTT**.
- Database: **MongoDB** and **Redis**.
- Secure Remote Access (Backend): **Tailscale Funnel**.
- Additional Tools: **Vite** (for development process).

Technical Justification: **Node.js (Express.js)** is used as the backend structure because of its property such as speed and real-time execution, easier integration with IoT protocols, and has lower runtime (it has high resource efficiency). **React.js** (and r3f) + Tailwind CSS is used as the frontend structure because it will leverage the component-based structure of react and at the same time be able to develop 3D web applications (including the usage of the .glb file later). **MongoDB** used for the device registry & metadata (device ID, device name, device type, or other data), any needed authentication & credentials (Local API tokens, MQTT credentials, or other credentials information), digital twin mapping (the physical device ID mapped to the 3D objects ID within the .glb file). **Redis** is used for short-term high-speed memory & communication usage [7], which include the storage of current device state (Example: device name as its key, power status as its value). **Tailscale Funnel** is used to expose the local backend to the public internet safely. For communication protocols detailed justification, it will be explained in **3.1.b Answer** below.

System Architecture Diagram:

M Ammar Hanif (13224087)

***Showroom Local Server Can't Accept BLE.**

**assume CCTV recording is directly streamed to the Web App!*

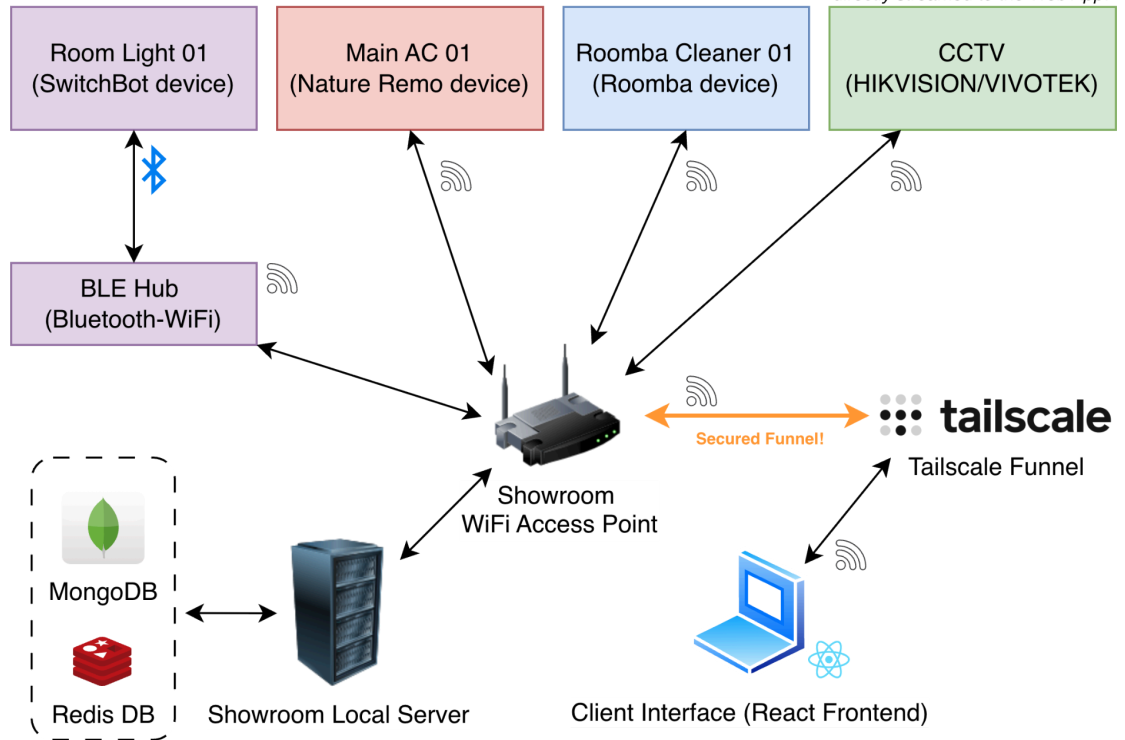


Figure 1. Smart Control Showroom Proposed System Architecture Diagram (Local Server Can't Accept BLE).

M Ammar Hanif (13224087)

**Showroom Local Server Can Accept BLE.*

**assume CCTV recording is directly streamed to the Web App!*

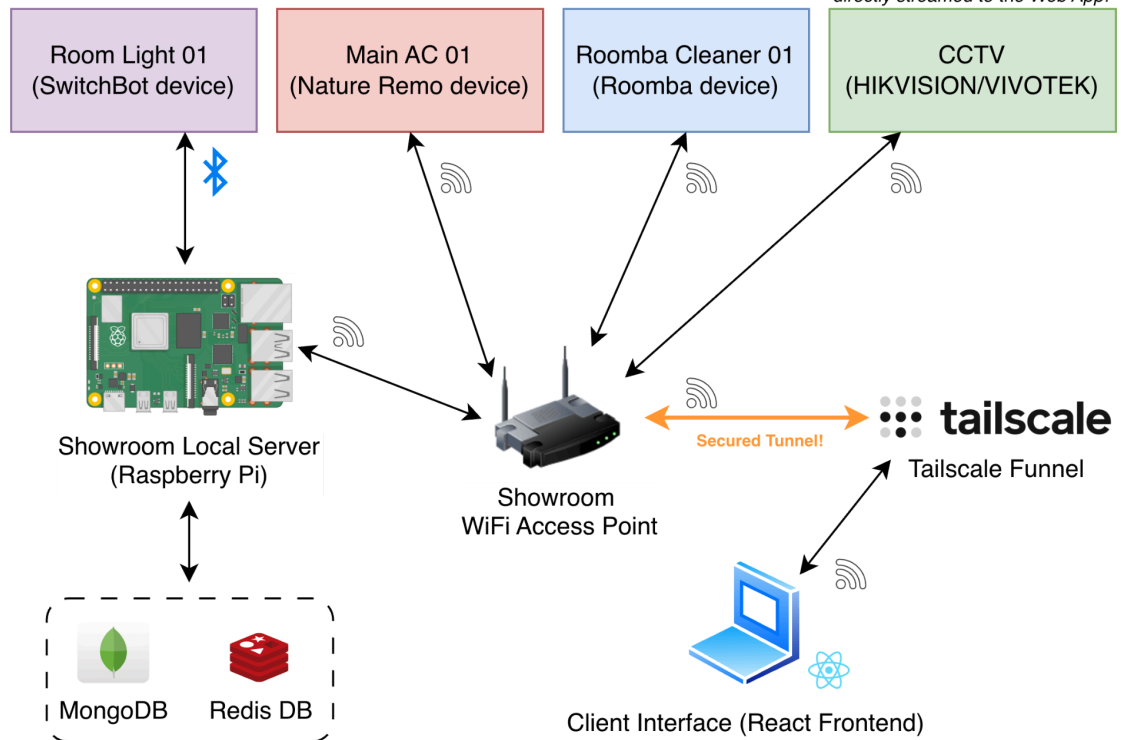


Figure 2. Smart Control Showroom Proposed System Architecture Diagram (Local Server Can Accept BLE, example: Raspberry Pi as Local Server).

b. Communication Protocols Comparison:

- Cloud API:
 - **When Used:** Remote access (off-LAN). This condition is used when Local API/BLE is unreachable (example: a mobile app is checking the showroom status from outside the area).
 - **Fallback Strategy:** If Cloud API call fails or hits the API rate limit, then serve last-known state from Redis storage (with a “stale” flag when the redis TTL expires). It won’t block the dashboard.
 - **Migration Strategy:** When the cloud quota is unavailable (or a request fails), then migrate to Local API/BLE if available. If it does not exist too, then display a specific status in the display until the cloud quota resets until the cloud quota resets.
- Local API:
 - **When Used:** Default status whenever backend and device are on the same LAN. This path will be set to be chosen first for lowest latency and no rate limit for the API call.
 - **Fallback Strategy:** If Local API times out (example: device rebooted), then fallback to the Cloud API automatically, log the failure, and retry

Local discovery on a backoff timer in the background (example: 60 second).

- **Migration Strategy:** When the fallback condition happens, then migrate to Cloud API and run a background retry attempt to rediscover Local API (example: every 60s). When successfully reconnects to the Local API, migrate back to Local API.
- BLE (Bluetooth-Low-Energy):
 - **When Used:** Backend/gateway is physically near the device.
 - **Fallback Strategy:** If connection fails due to range/RSSI weak signal, then switch to Cloud API (only if device has a Hub bridging it to the cloud). If no Hub exists, then the device will be shown unreachable.
 - **Migration Strategy:** BLE is only re-attempted on the next user-issued command, because range issues are physical/positional and won't be resolved based on a timer. So there will be no background retry.
- MQTT/Webhook:
 - **When Used:** For event-driven local devices (such as Roomba) that push state changes and no cloud polling. Webhook is used as an optional Cloud-side push alternative, such that it minimises the cloud quota issues. For specific cases, for Roomba's device, it will use a local MQTT broker (same WiFi network with the backend).
 - **Fallback Strategy:** If local MQTT connection to Roomba drops, then fallback to *iRobot Cloud API* [3]. In this case, it will use the iRobot Cloud API with a cloud server.
 - **Migration Strategy:** When disconnected from the Local MQTT broker, it will migrate to the iRobot Cloud API connection. The backend will keep trying to reconnect to the Local MQTT broker in the background (example: every 60s). If it successfully reconnects to the Local MQTT, then migrates back to the Local MQTT connection.

c. Dashboard Command Pipeline:

Command Pipeline Diagram Assumption Available Data & State (Action):

- Main Light 01
 - Turn OFF
 - Turn ON
- Main TV 01
 - Turn OFF
 - Turn ON
- Main Roomba 01
 - Turn OFF
 - Turn ON

Complete Simple Web App Solution: [Github Repo](#) [Demo Video](#)

Complete Detail System Explanation: [README](#)

Pipeline Order:

- **Flow 1: Digital Twin → Physical Device**

1. User clicks a control (e.g. "Turn ON") on the React App.
2. Frontend sends `POST /api/devices/command` with `{ deviceID, command, currentState }`
3. `deviceController.js` receives the request, calls `commandRouter.js`
4. `commandRouter.js` looks up the device's protocol in MongoDB.
5. Router delegates to the matching adapter (`switchBotAdapter / natureRemoAdapter / roombaAdapter`).
6. Adapter executes the vendor-specific command against the physical device.
7. On success, the new state is written to Redis.
8. Redis fires a keyspace notification on that key change.
9. `socket.js` listener catches the notification, reads the updated value, and send the newest update of the Redis data via Socket.IO.
10. Every connected browser session (physical device card + digital twin card) receives the event and updates its UI simultaneously.

- **Flow 2: Physical Device → Digital Twin**

This covers the case where a device's real state changes independently, keeping the Digital Twin accurate all the times. For this demonstration, this flow is triggered via the Physical Devices Control (Simulator) interface.

For MQTT devices (Roomba):

1. A physical change is simulated by clicking a control on the frontend Simulator.
2. Frontend sends `POST /api/webhooks/simulate` with `{ deviceID, state }`.
3. The webhook controller acts as the physical hardware and publishes a real MQTT payload to a unique topic (example: `nakayama/roomba/{deviceID}/state`) on the public test broker.
4. `mqttSubscriber.js`, which maintains a persistent connection to that same broker, receives the published message.
5. The subscriber parses the hardware payload, merges it with any unchanged state data, and writes the new state to Redis.
6. Redis fires a keyspace notification on that key change (key process to making sure the Digital Twin is always updated).
7. `socket.js` listener catches the notification, reads the updated value, and send the newest update of the Redis data via Socket.IO.
8. Every connected browser session receives the event and updates its 3D Digital Twin simultaneously.

For Cloud API / Local API devices (SwitchBot / Nature Remo):

1. A physical change is simulated by clicking a control on the frontend Simulator.
2. Frontend sends `POST /api/webhooks/simulate` with `{ deviceID, state }`.
3. The webhook controller acts as an incoming webhook push from the vendor, bypassing external routing and directly writing the new state to Redis.
4. Redis fires a keyspace notification on that key change.
5. `socket.js` listener catches the notification, reads the updated value, and send the newest update of the Redis data via Socket.IO.
6. Every connected browser session receives the event and updates its 3D Digital Twin simultaneously.

Command Pipeline Diagram in form of Data Flow Diagram:

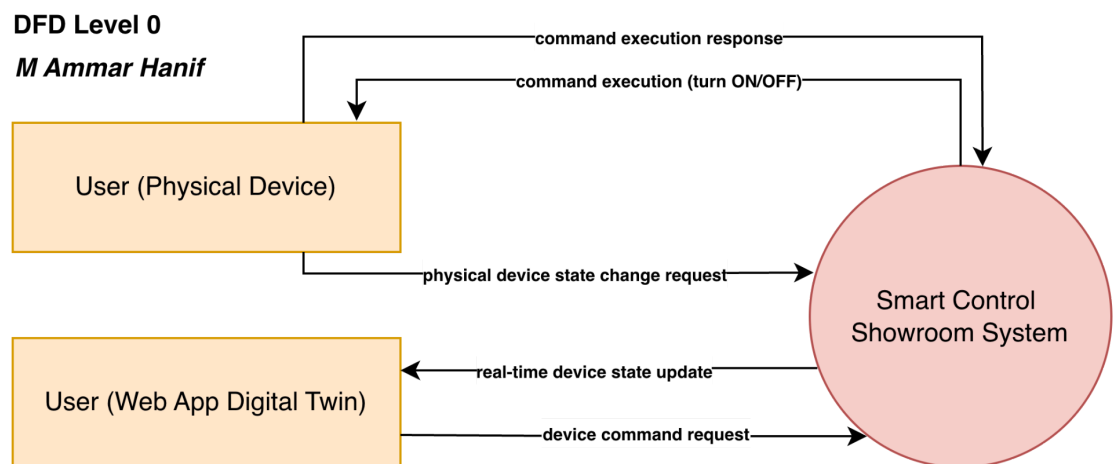


Figure 3. Dashboard Command Pipeline Diagram (DFD Level 0).

DFD Level 1

M Ammar Hanif

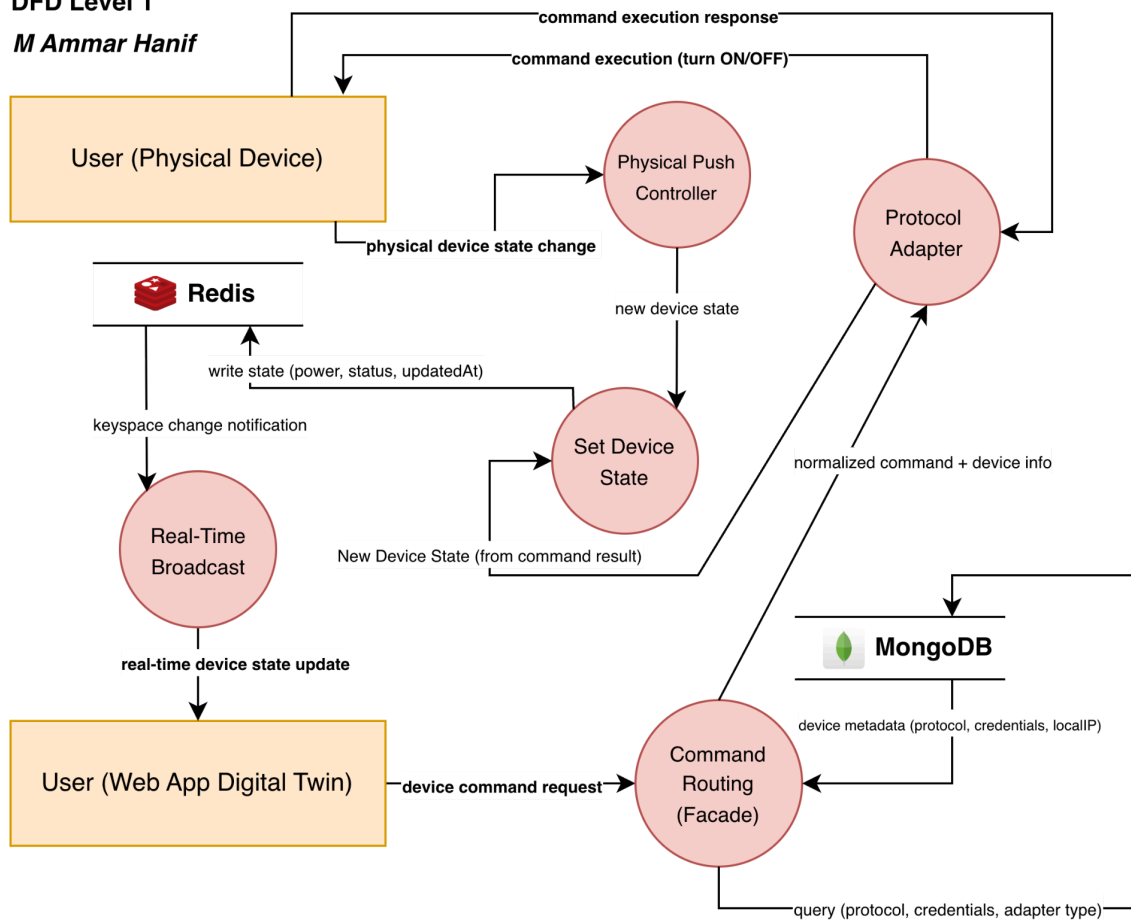


Figure 4. Dashboard Command Pipeline Diagram (DFD Level 1).

- d. Integrating multiple vendors CCTV Cameras into a single browser-based viewer:
 The core problem is that every CCTV vendor uses a different URL format and connection details by default. Without a standard, we have to manually hard-code it one by one, and it's not efficient. Thus, we need a standardizer tool to standardize the format. For this tool, we can use **RTSP**, which is a standardized protocol for many CCTV vendors. CCTV/Cameras that support RTSP will expose a standard API that later will be consumed by the **go2rtc Streaming/Media Server**.

Technology Stack Needed for Multiple-Vendor CCTV Setup:

- Streaming server: **go2rtc (go to Real Time Communication)**
- Standardization/normalization: **RTSP (Real Time Streaming Protocol)**
- Secure Remote Access: **Tailscale Funnel**
- Frontend: **React App** (match with the previous 3.1.c answer setup) and **Standard iframe** HTML component.

Technical Justification: **go2rtc** is used as the media/streaming server because it acts as a universal protocol converter, seamlessly transforming RTSP camera feeds into WebRTC with extremely low latency [8]. **RTSP** is used because it provides a low

latency stream and native support on RTSP cameras [9]. **Tailscale Funnel** is used to expose the local backend (and by extension for **go2rtc**) and the frontend (for demo purposes) to the public internet safely. **Standard iframe** component is used to minimize any additional system-down risk in the process of displaying the stream to the interface.

Multiple Vendor CCTV Architecture Diagram:

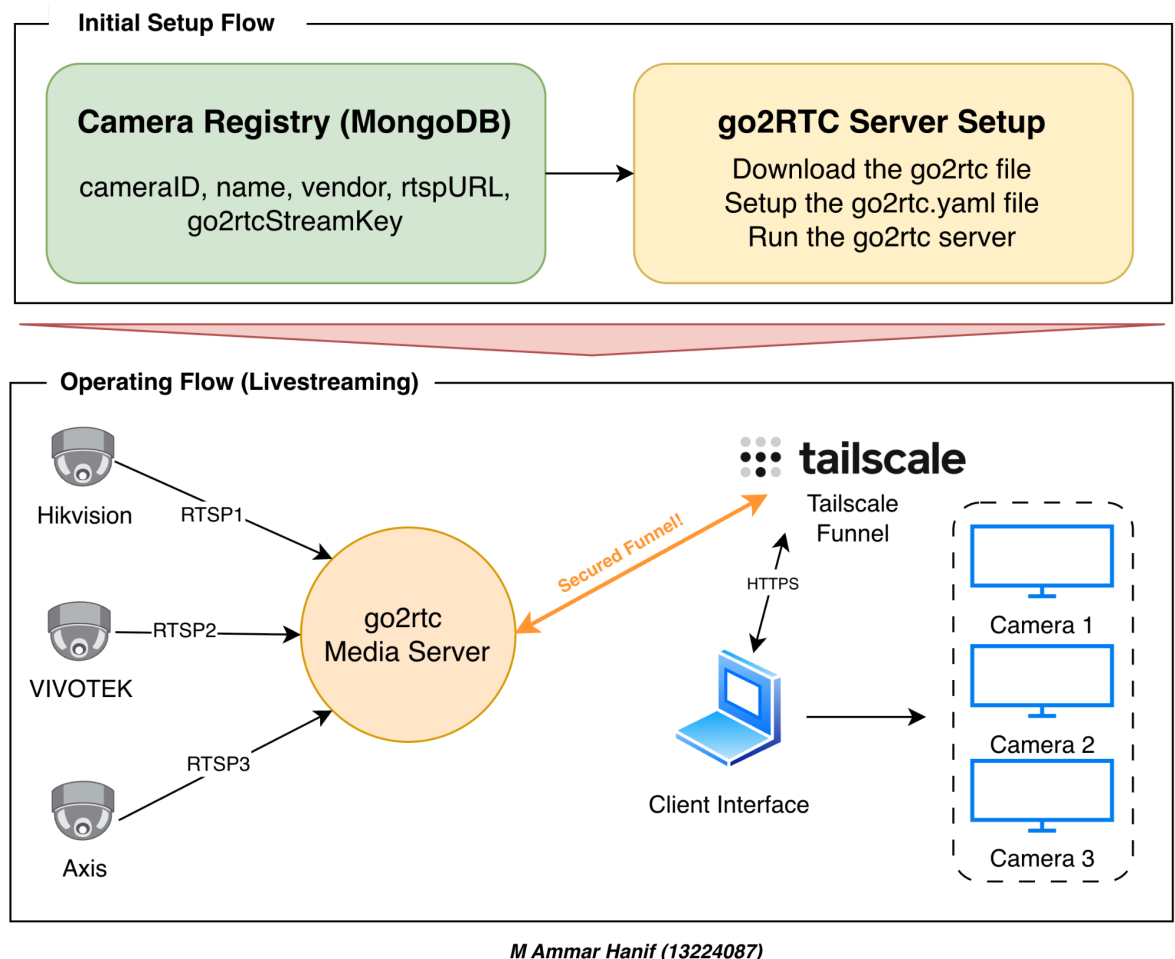


Figure 5. Multiple Vendor CCTV Architecture Diagram.

3.2 3D Digital Twin Dashboard, Integrate a pre-built GLB showroom model into the same

dashboard with real-time device synchronization.

- 3D Stack & Architecture Showing How Backend State Updates the 3D Scene:
Based on my answer in 3.1.a, I propose that the frontend will be using **React.js**, and for the 3D requirements, it will be using **react-three-fiber**, which include:
 - Three.js:** The core of WebGL rendering engine.
 - react-three-fiber (r3f):** The React renderer for Three.js

- **@react-three/drei's:** useGLTF + GLTFLoader, which loads the .glb model later. [6]
- **DRACOLoader:** decompresses Draco-compressed geometry. [6]
- **OrbitControls (via drei):** a Camera navigation (to pan/zoom/rotate) for the operator to “move around” inside the model. [6]

Technical Justification: React.js and mainly **react-three-fiber (r3f)** is used as the frontend structure because it will leverage the component-based structure of react and at the same time be able to develop 3D web applications (including the usage of the .glb file later).

3D View Generation Architecture Diagram:

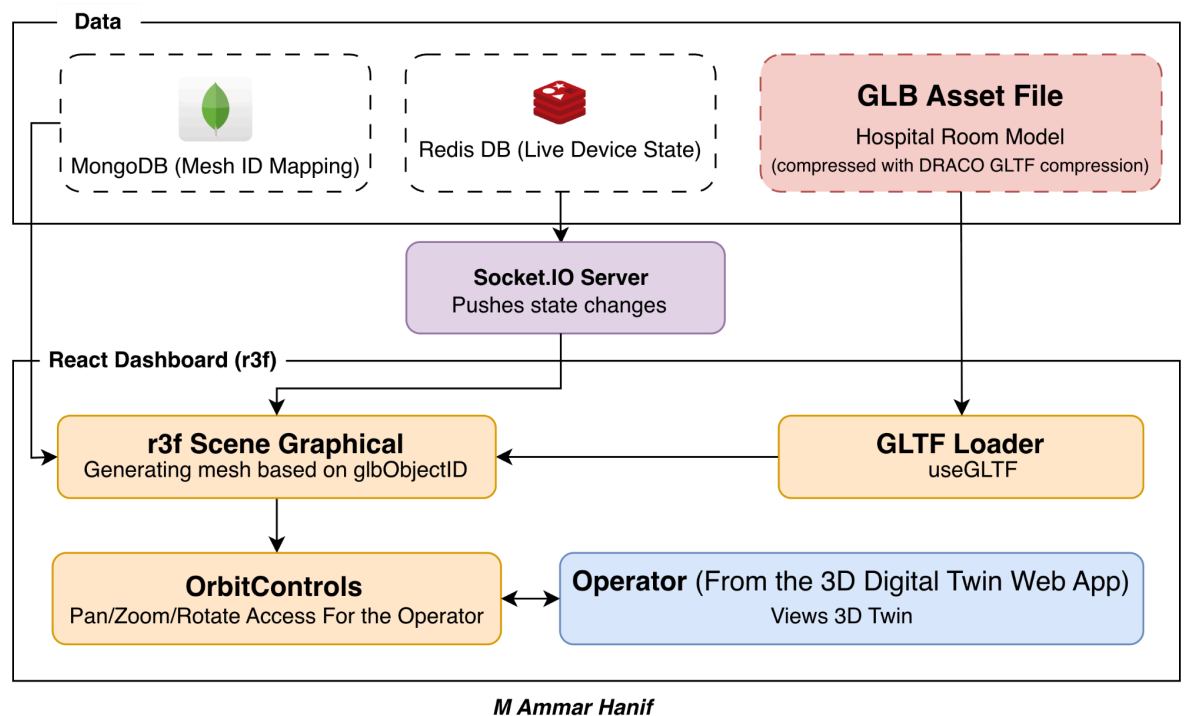


Figure 6. Backend and 3D Frontend Architecture Diagram.

- b. Simple Pipeline Demonstrating Backend State Update Reflected in the 3D Scene:
Complete Web App Solution: [Github Repo](#) [Demo Video](#)

Complete Detail System Explanation: [README](#)

Pipeline Order:

It adopt the same flow of the previous answer. Please refer to [Flow 1](#) and [Flow 2](#) previously

Web App Interface:

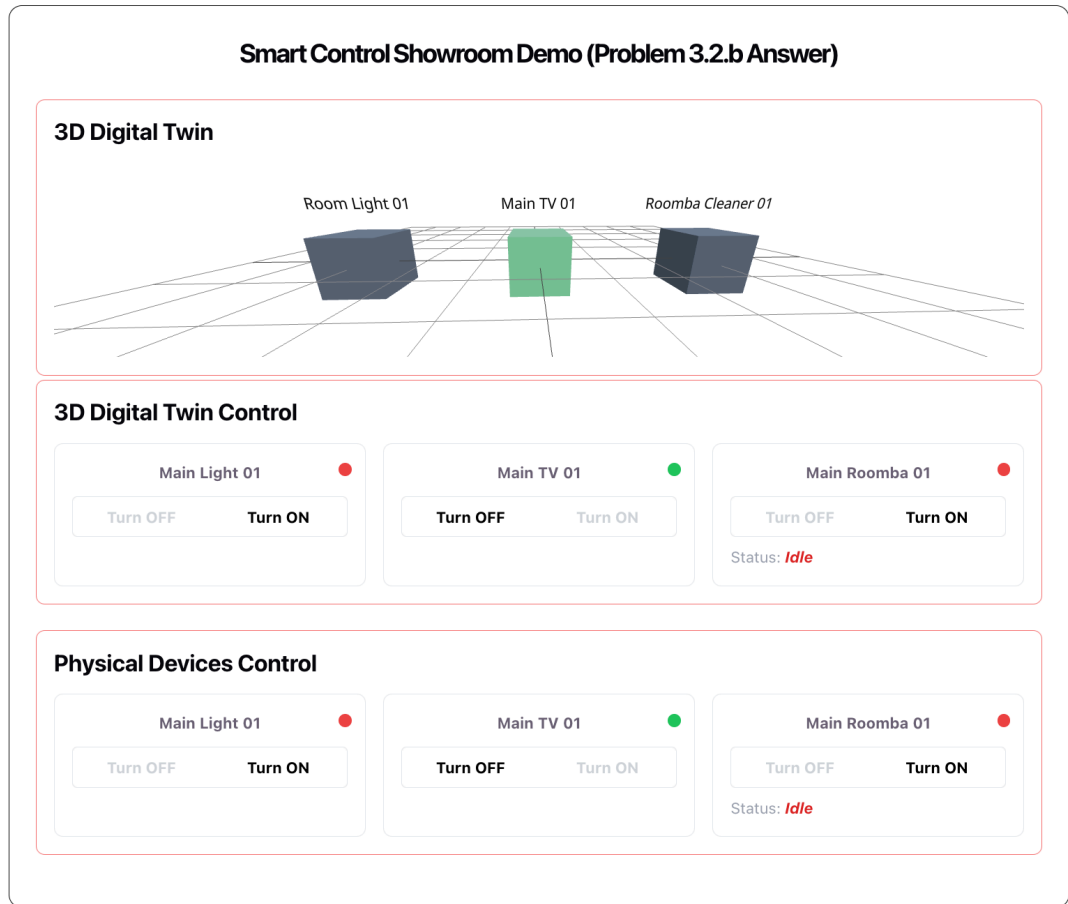


Figure 7. Simple 3D Digital Twin Web App Implementation.

Backend → 3D Scene Pipeline Flow Explanation:

`Scene3D.jsx` renders the given `.glb` file. The objects that we are able to control are the light (`Main_Light_01`), TV (`Main_TV_01`), and Roomba (`Main_Roomba_01`), each subscribed to the same states object already provided by `useDeviceSocket.js`. This component consumes the exact same device state change event, which fully explained orderly in [Flow 1](#) and [Flow 2](#) previously.

- c. Simple Dashboard Combining The Device Into Stack:
Complete Web App Solution: [Github Repo](#) [Demo Video](#)

Complete Detail System Explanation: [README](#)

Pipeline Order:

It adopt the same flow of the previous answer. Please refer to [Flow 1](#) and [Flow 2](#) previously

Web App Interface:

 **Nakayama**

IoT - PLC Team Case Study - ITB de Labo 2026 Recruitment

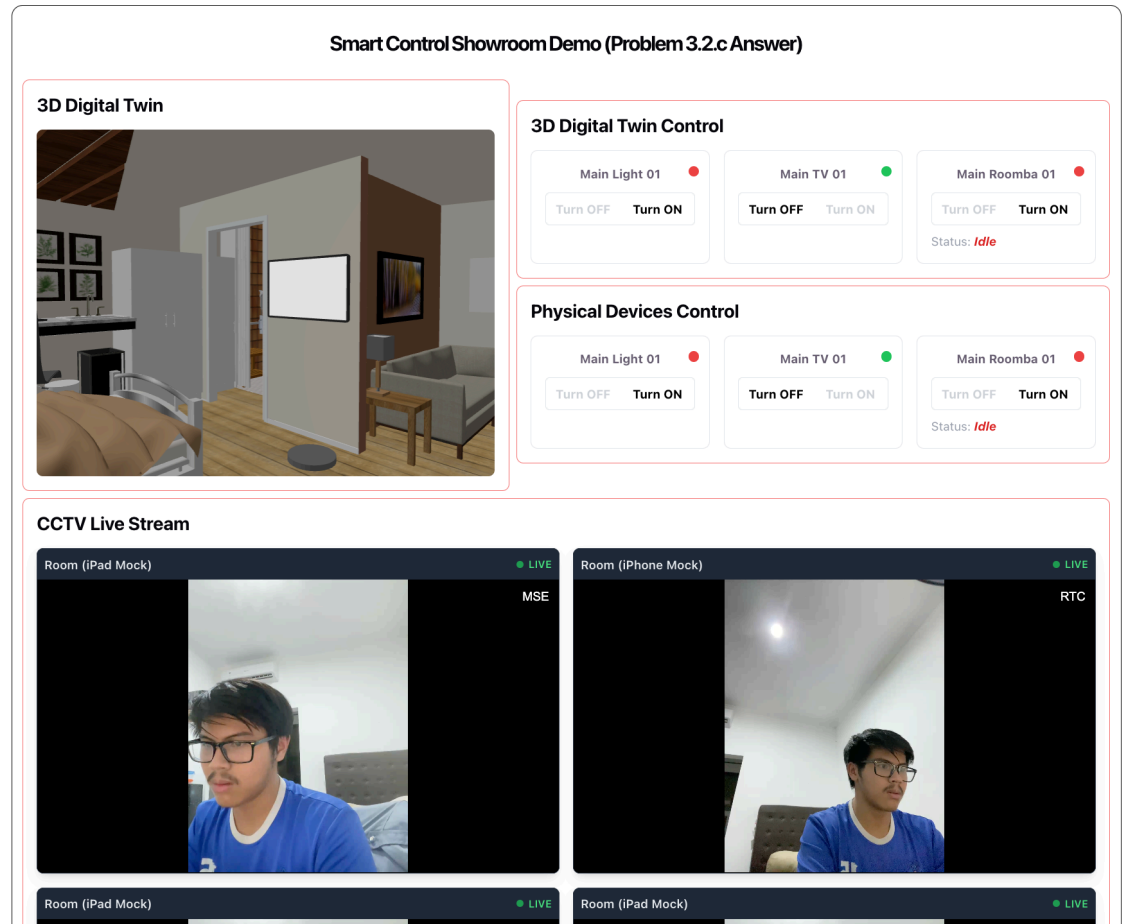


Figure 8. Simple 3D Digital Twin Web App Complete Implementation.

References

- [1] OpenWonderLabs, "SwitchBotAPI," GitHub. <https://github.com/OpenWonderLabs/SwitchBotAPI> (accessed Jul. 14, 2026).
- [2] Nature, "nature-remo," GitHub. <https://github.com/nature-remo/nature-remo> (accessed Jul. 14, 2026).
- [3] koalazak, "dorita980," GitHub. <https://github.com/koalazak/dorita980> (accessed Jul. 14, 2026).
- [4] GeeksforGeeks, "Facade Design Pattern - Introduction," GeeksforGeeks. <https://www.geeksforgeeks.org/system-design/facade-design-pattern-introduction/> (accessed Jul. 14, 2026).
- [5] Evon Technologies, "Why use Node.js development for IoT applications? Top reasons," Evon Technologies. <https://evontech.com/component/easyblog/why-use-node-js-development-for-iot-applications-top-reasons.html?Itemid=159?tmpl=component&print=1&format=print> (accessed Jul. 14, 2026).
- [6] Poimandres, "Introduction - Drei," pmndrs docs. <https://drei.docs.pmnd.rs/getting-started/introduction> (accessed Jul. 14, 2026).
- [7] Redis, "Redis Pub/Sub," Redis Documentation. <https://redis.io/docs/latest/develop/pubsub/> (accessed Jul. 14, 2026).
- [8] WebRTC.link, "go2rtc: Ultimate streaming solution," WebRTC.link. <https://webrtc.link/en/articles/go2rtc-ultimate-streaming-solution/> (accessed Jul. 14, 2026).
- [9] Red5, "4 Reasons RTSP Streaming is Still Relevant," Red5. <https://www.red5.net/blog/4-reasons-rtsp-streaming-is-still-relevant/> (accessed Jul. 14, 2026).

Appendices

Appendix 1. Smart Control Showroom Github 3.2 Solution Repository.

https://github.com/hanif-page/Smart-Control-Showroom_13224087

Appendix 2. Devices in Smart Control Showroom.

Device Brand	Candidate Protocol	Object
SwitchBot	Cloud API / BLE / Matter	Lamp, Humidity, Bot, and Hub

Nature Remo	Cloud API / Local API	TV and AC
Roomba	MQTT / Local REST API	Roomba Robot Cleaner
HIKVISION/VIVOTEK	RTSP / ONVIF / Web RTC	CCTV

Appendix 3. Example of 3D model for digital twin and architecture of Nakayama.

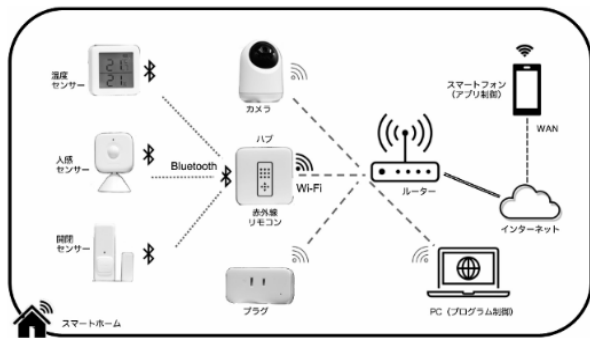
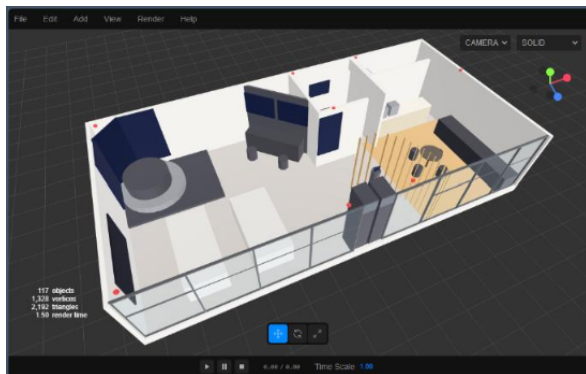
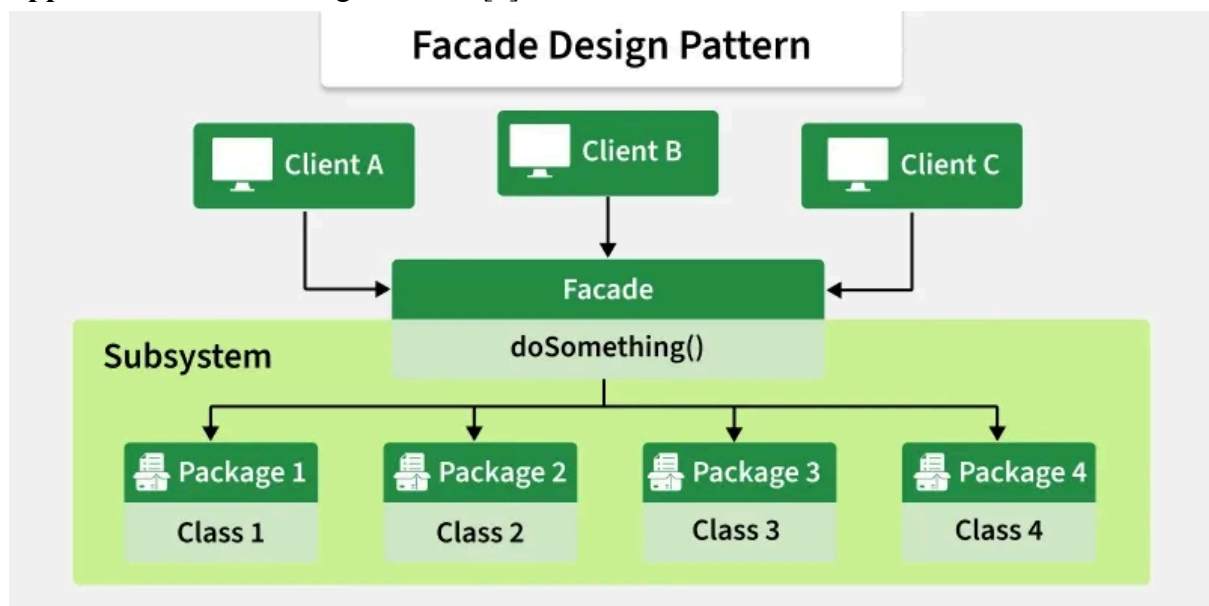
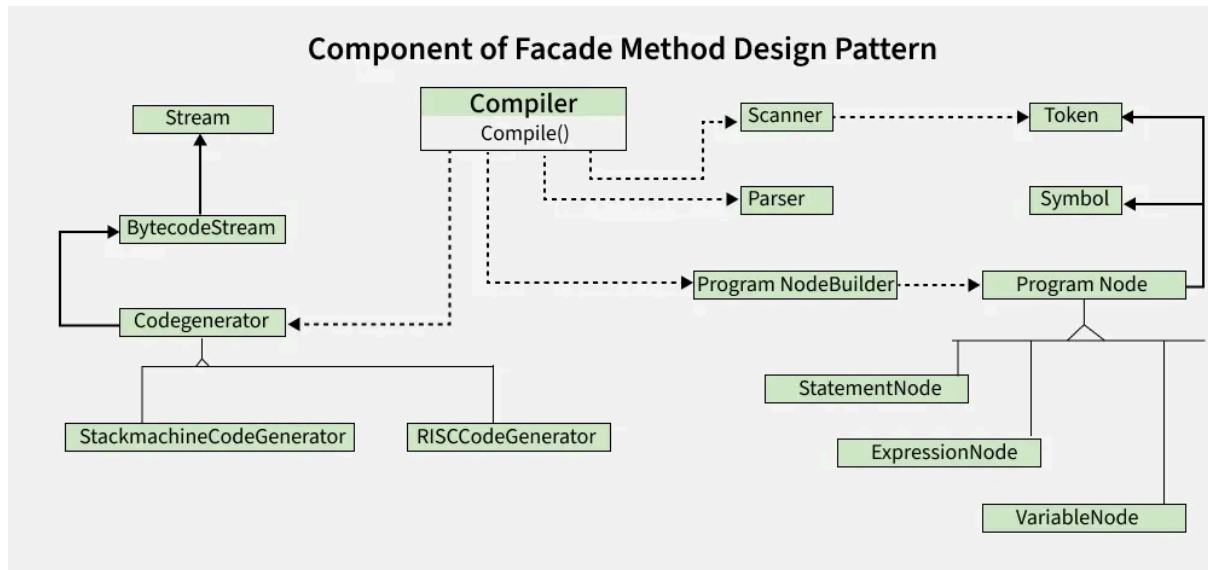


図 1-1 IoT デバイスを使ったスマートホームのネットワーク構成図

Appendix 4. Facade Design Pattern. [4]



Appendix 5. Example of Component of Facade Method Design Pattern. [4]



Appendix 6. Final & Complete Demonstration Video (Report & Final Web App).

https://drive.google.com/file/d/17y_mdCRaBiCSgBMaK1Z9le9QxE--eeMH/view?usp=sharing

Appendix 7. 3.1.c Web App Result Demonstration.

https://drive.google.com/file/d/1nJsGcD1Xi_gHnBqtVVs790pVCLjQFAMN/view?usp=sharing

Appendix 8. 3.2.b Web App Result Demonstration.

https://drive.google.com/file/d/1WueDMv9ddOkEfl_wK2lAnxorbtRPdTL1/view?usp=sharing

Appendix 9. 3.2.c Web App Result Demonstration.

<https://drive.google.com/file/d/1giUE4374Iv0AmEewBKTBCG8F481vPmaK/view?usp=sharing>